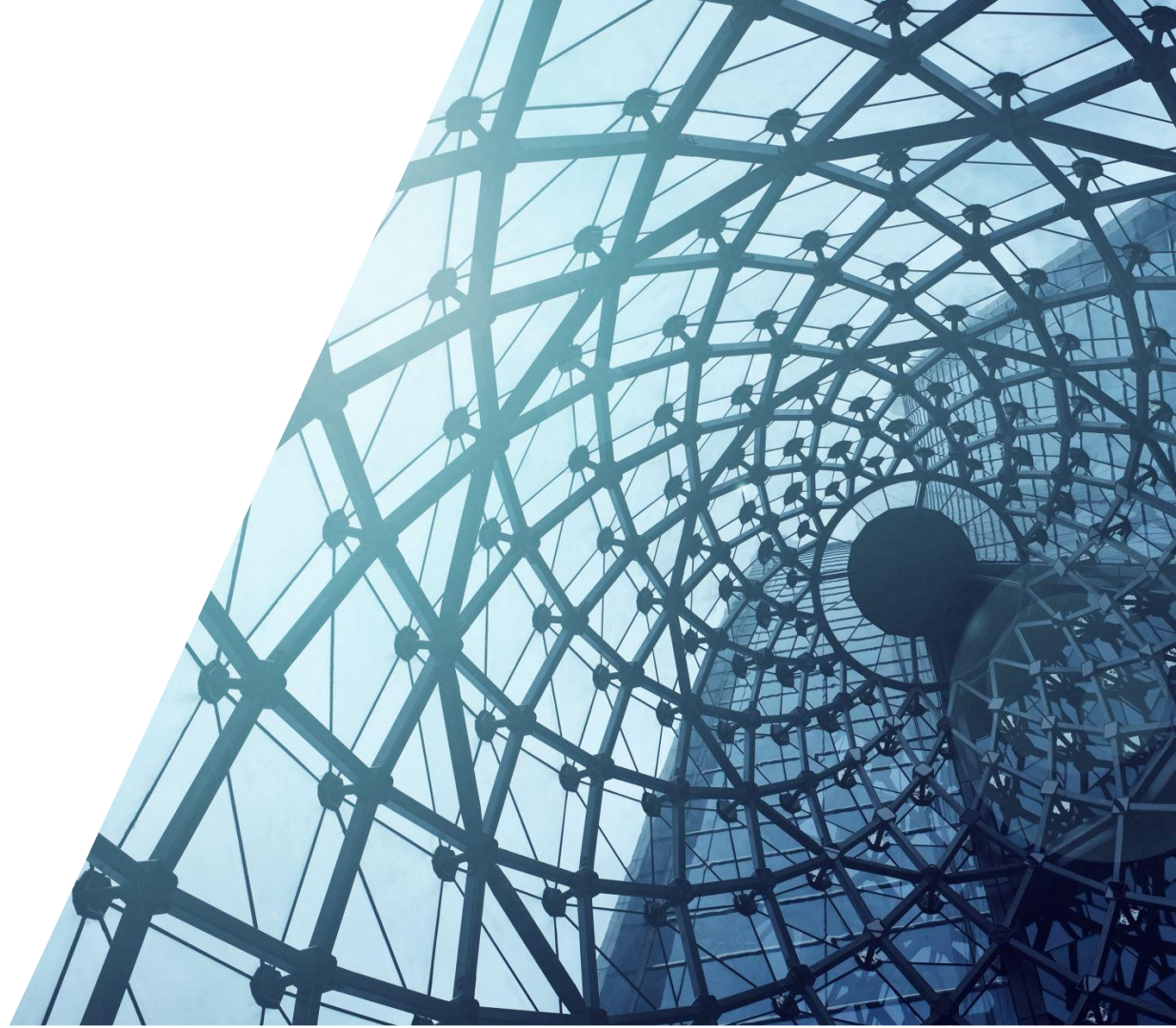
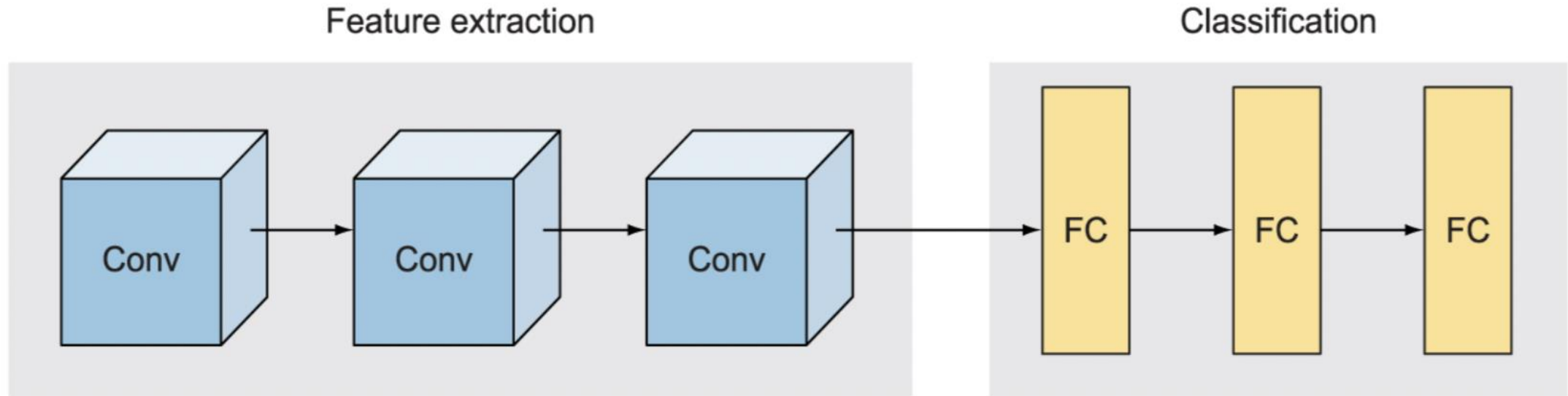


Advanced CNN Architecture

Lecture 11



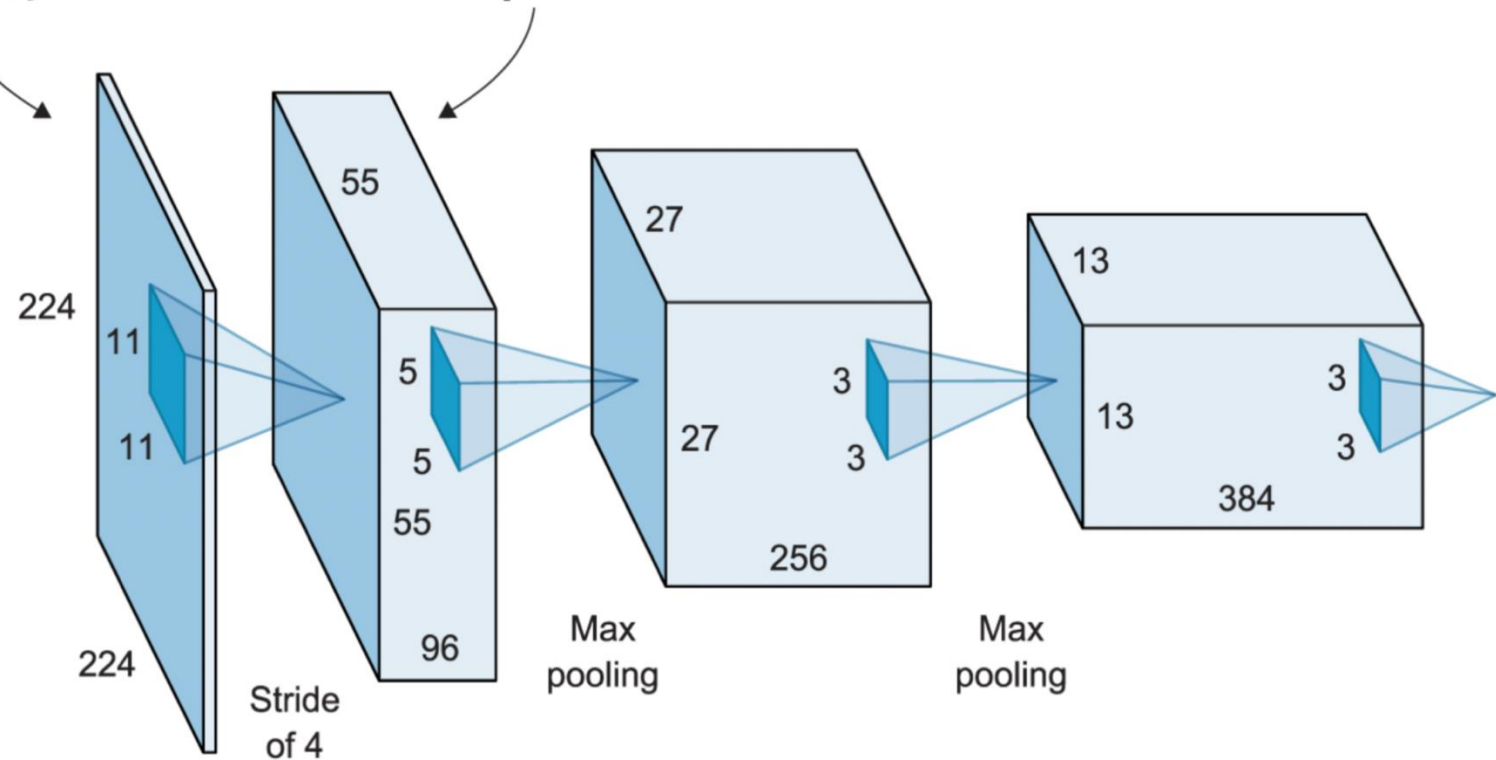
CNN Design Patterns



CNN Design Patterns – Feature Extraction

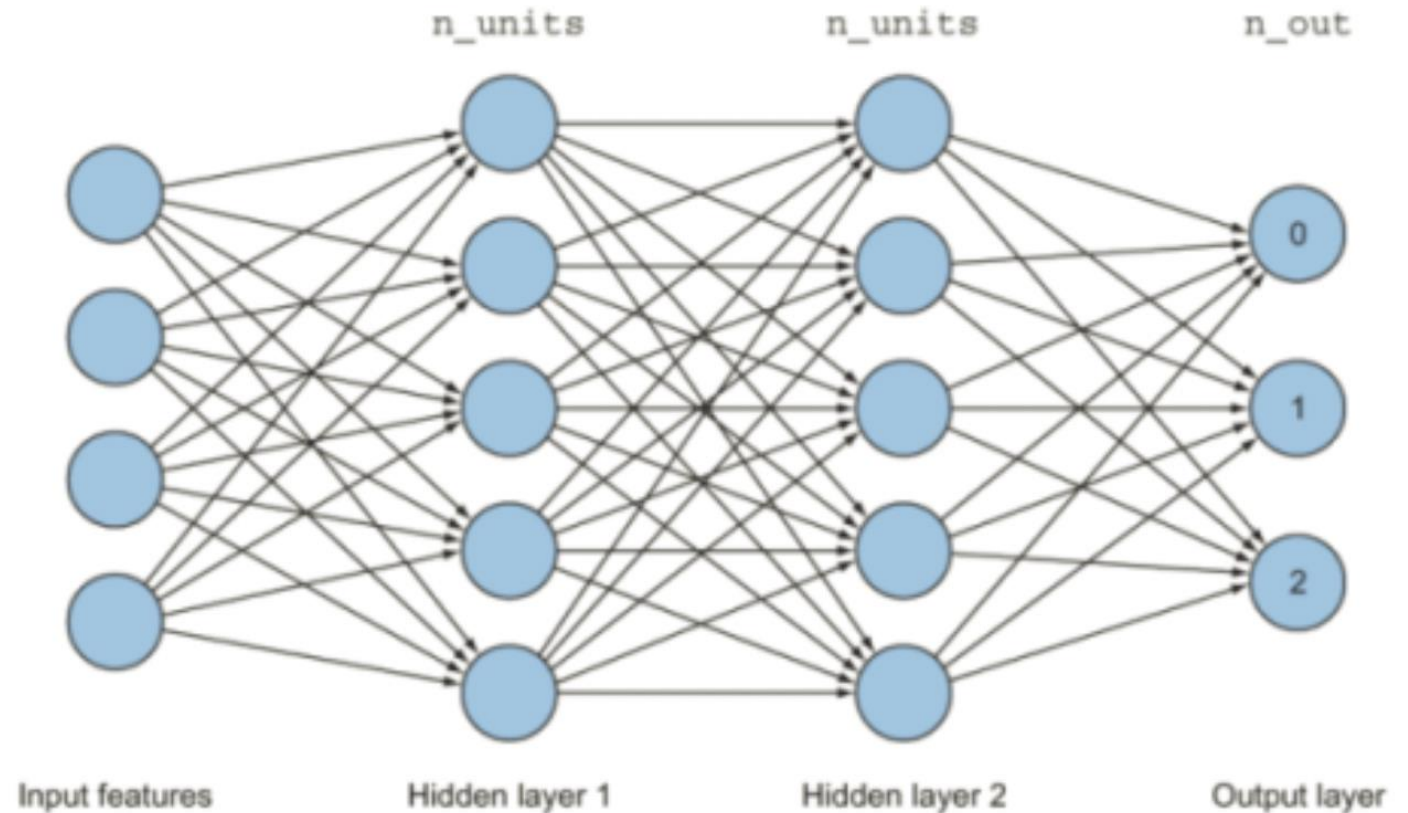
Input image = $H \times W \times \text{channel}$ Image volume = $H \times W \times \text{feature maps}$
Channel = {R, G, B} = 3 Feature maps = 96

- Depth = # channels for the input image
- Depth = # feature maps for the hidden layers
- Pattern: As we go deeper in the network, image depth increases, dimensions decrease



CNN Design Patterns – Fully Connected Layers

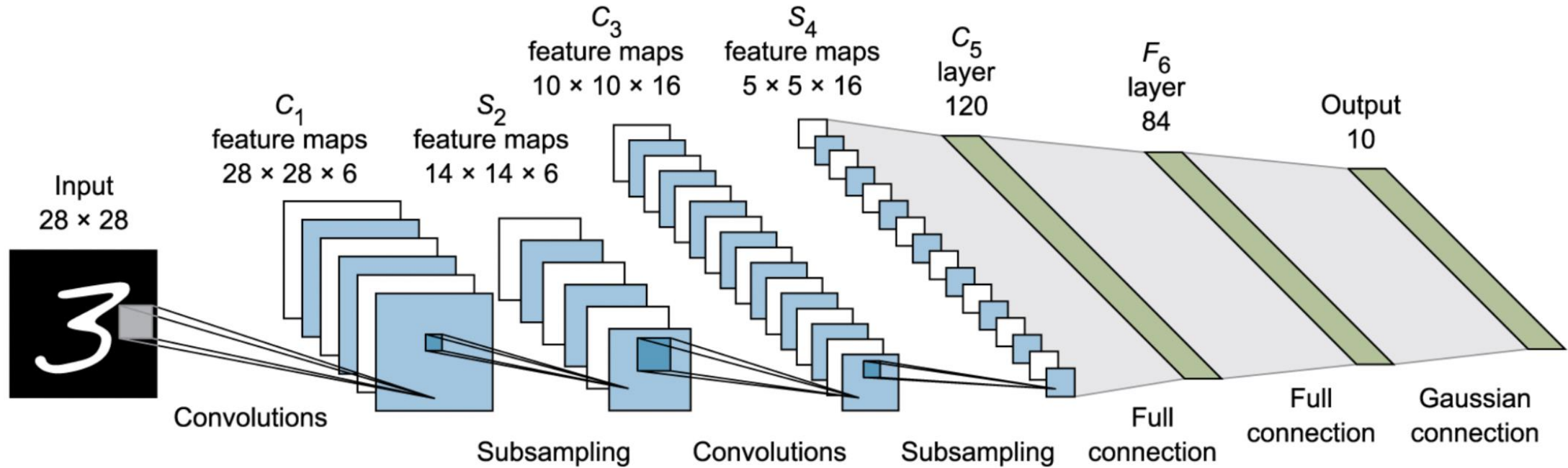
- Typically, all fully connected layers have same number of hidden units or decrease at each layer
- In order to limit the design choices, the number of hidden units in the fully connected layers can be kept constant



LeNet-5

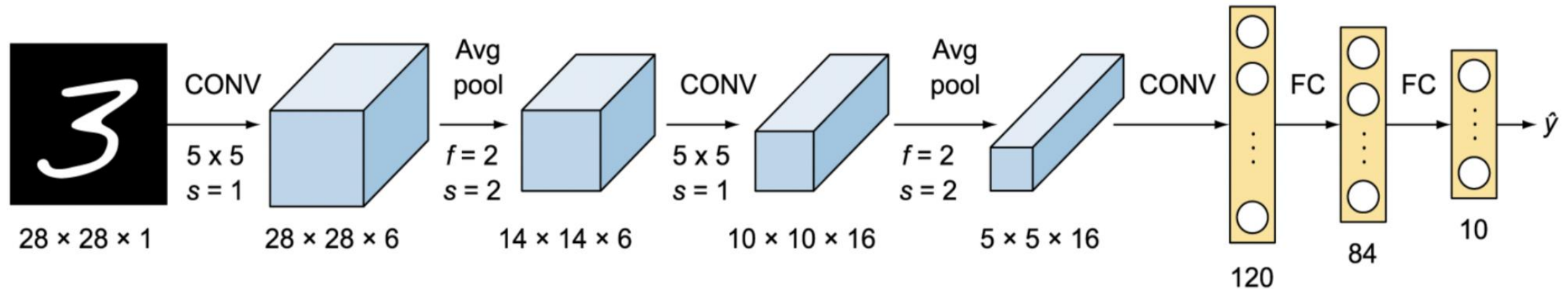
- Architected by Lecun et. al. in 1998[1]
- 99% accuracy on MNIST
- Network depth = 5
 - 3 convolutional layers + 2 fully connected layers
 - Also known as weight layers since they contain trainable weights
 - Pooling does not contain trainable weights
 - Reflects on computational complexity of the model

LeNet-5 Architecture

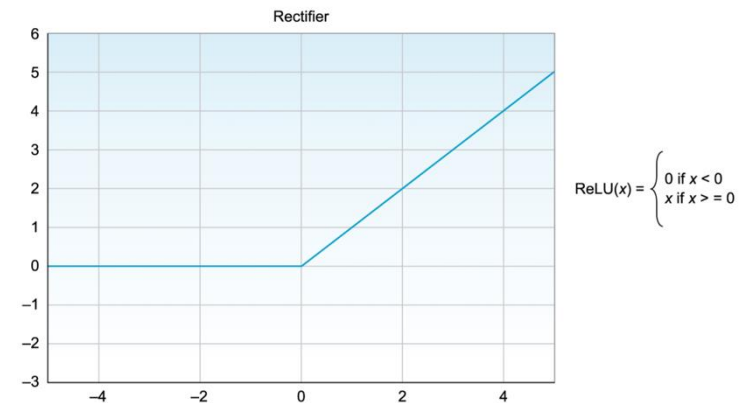
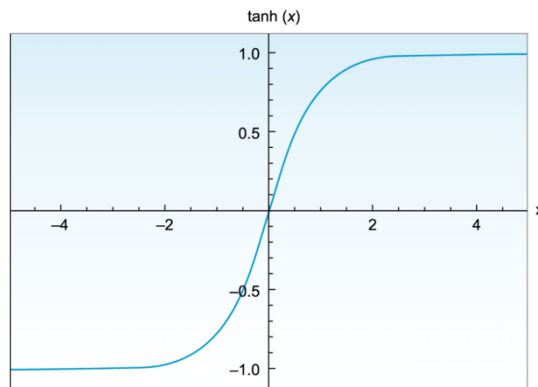


- INPUT IMAGE $\Rightarrow$ C₁ $\Rightarrow$ TANH $\Rightarrow$ S₂ $\Rightarrow$ C₃ $\Rightarrow$ TANH $\Rightarrow$ S₄ $\Rightarrow$ C₅ $\Rightarrow$ TANH $\Rightarrow$ FC₆ $\Rightarrow$ SOFTMAX₇

LeNet-5 Implementation



- Filters = C1 $\rightarrow$ 6, C1 $\rightarrow$ 16, C1 $\rightarrow$ 16
- Filter size = 5×5
- Subsampling layer = Pooling layer $\rightarrow$ Average pooling
- Activation function = tanh



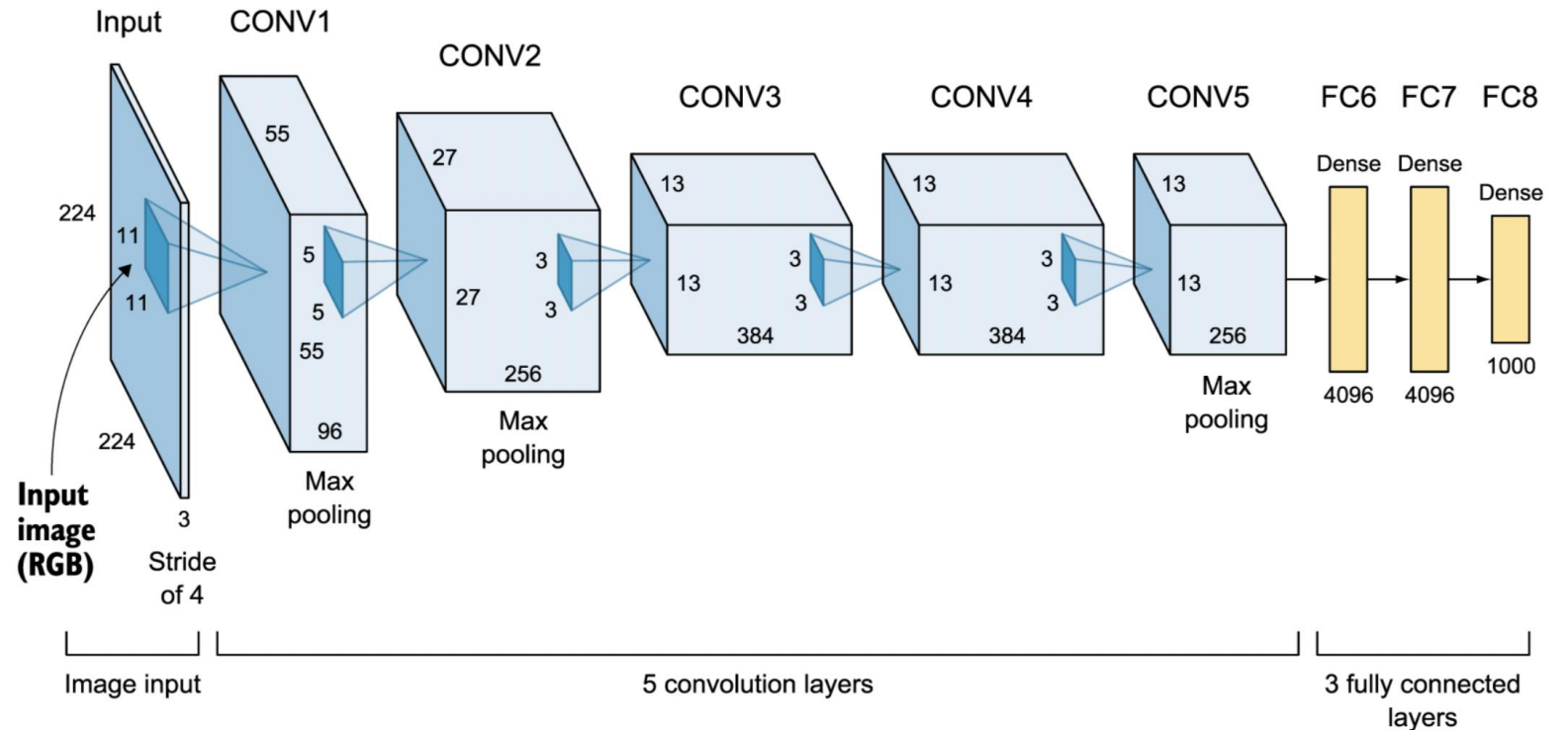
Setting up learning hyper-parameters

- Total trainable parameters for LeNet-5 = 61,706
- Scheduled decay learning rate with 20 epochs
 - `def lr_schedule(epoch):`
 - `if epoch <= 2:`
 - `lr = 5e-4`
 - `elif epoch > 2 and epoch <= 5:`
 - `lr = 2e-4`
 - `elif epoch > 5 and epoch <= 9:`
 - `lr = 5e-5`
 - `else:`
 - `lr = 1e-5`
 - `return lr`

lr is 0.0005 for the first two epochs, 0.0002 for the next three epochs (3 to 5), 0.00005 for the next four (6 to 9), then 0.00001 thereafter (more than 9).

AlexNet [2]

- Depth = weight layers = 8
- Trained on RGB images from ImageNet dataset
- Use of Relu Activation function – trains much faster
- Use of Max pooling
- Use of Drop out layers to avoid over fitting
- Trainable parameters = 60 million



AlexNet hyperparameters and Performance

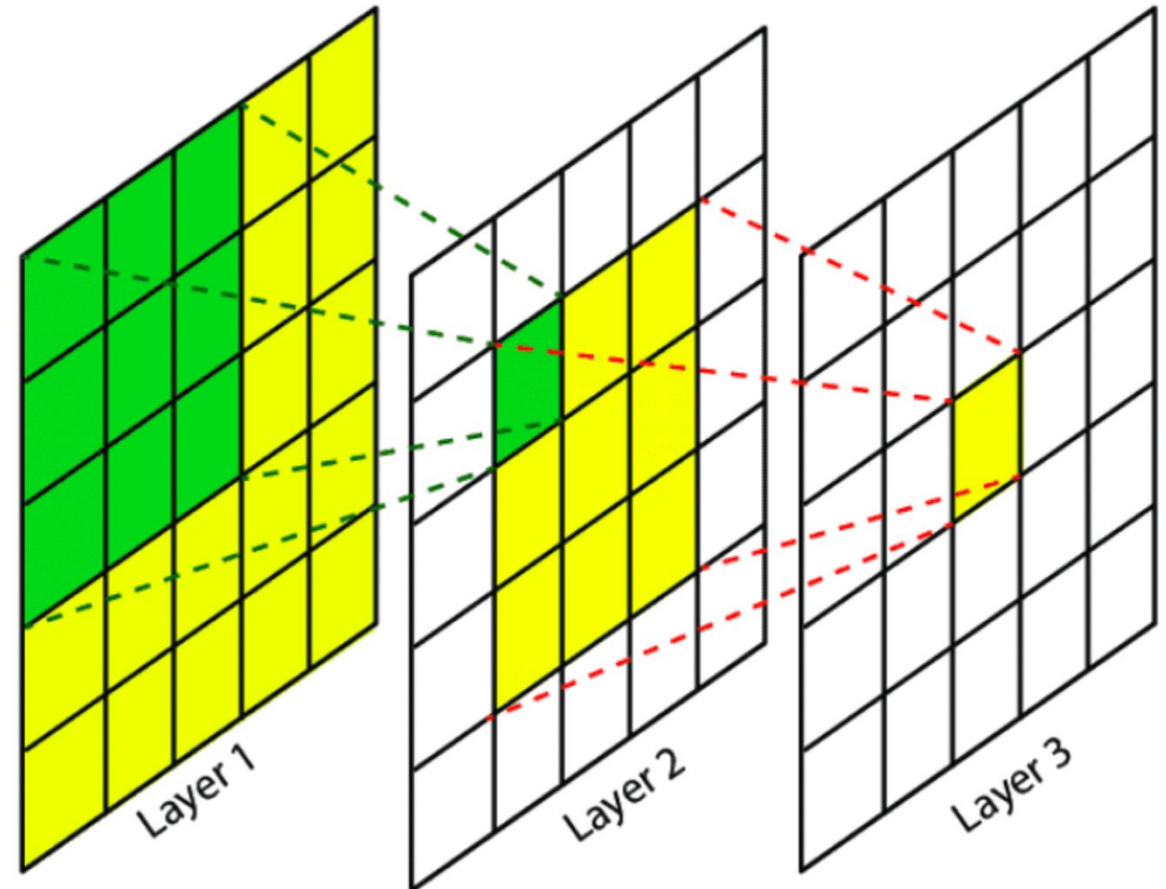
- Trained with 90 epochs
- Initial learning rate 0.01 with momentum 0.9
 - Reduces the learning rate as $lr/10$ when validation error stops improving
- Performance
 - Top-1 error rate - % of the time when classifier didn't give the correct class highest score
 - Top-5 error rate - % of the time when the classifier didn't include the correct class show among its top 5 guesses
 - AlexNet – 15.3%

VGGNet

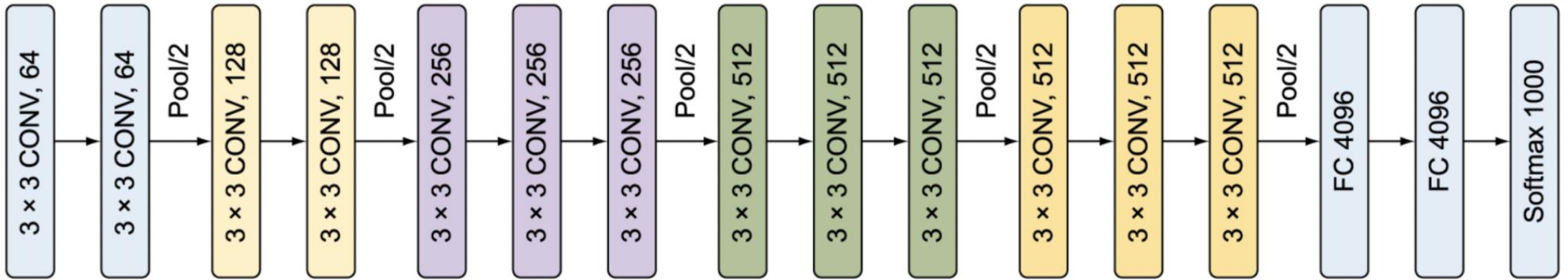
- Developed in 2014 by Visual Geometry Group at Oxford University
- 16 weight layers : 13 convolutional layers + 3 fully connected layers
- Simple architecture with uniform components
- Convolutional layers: 3x3 filters, stride =1, padding =same
- Pooling: Max pooling 2x2, stride 2
- Multiple stacked smaller filters are better than a single larger filter
 - Gives more depth to the model
 - Can extract more complex features

Receptive field

- 2 3×3 filters have effective receptive field as 5×5
- 3 3×3 filters have effective receptive field as 7×7
- More number of smaller filters use ReLu function multiple times to add non-linearity. That makes the decision function more discriminative
- Decreases number of training parameters due to the size of the filter



VGG-16 Architecture



VGG Architecture configuration

ConvNet configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
Input (224 x 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					

Network D = VGG16
Network E = VGG19

Table 5.1 VGGNet architecture parameters (in millions)

Network	A, A-LRN	B	C	D	E
No. of parameters	133	133	134	138	144

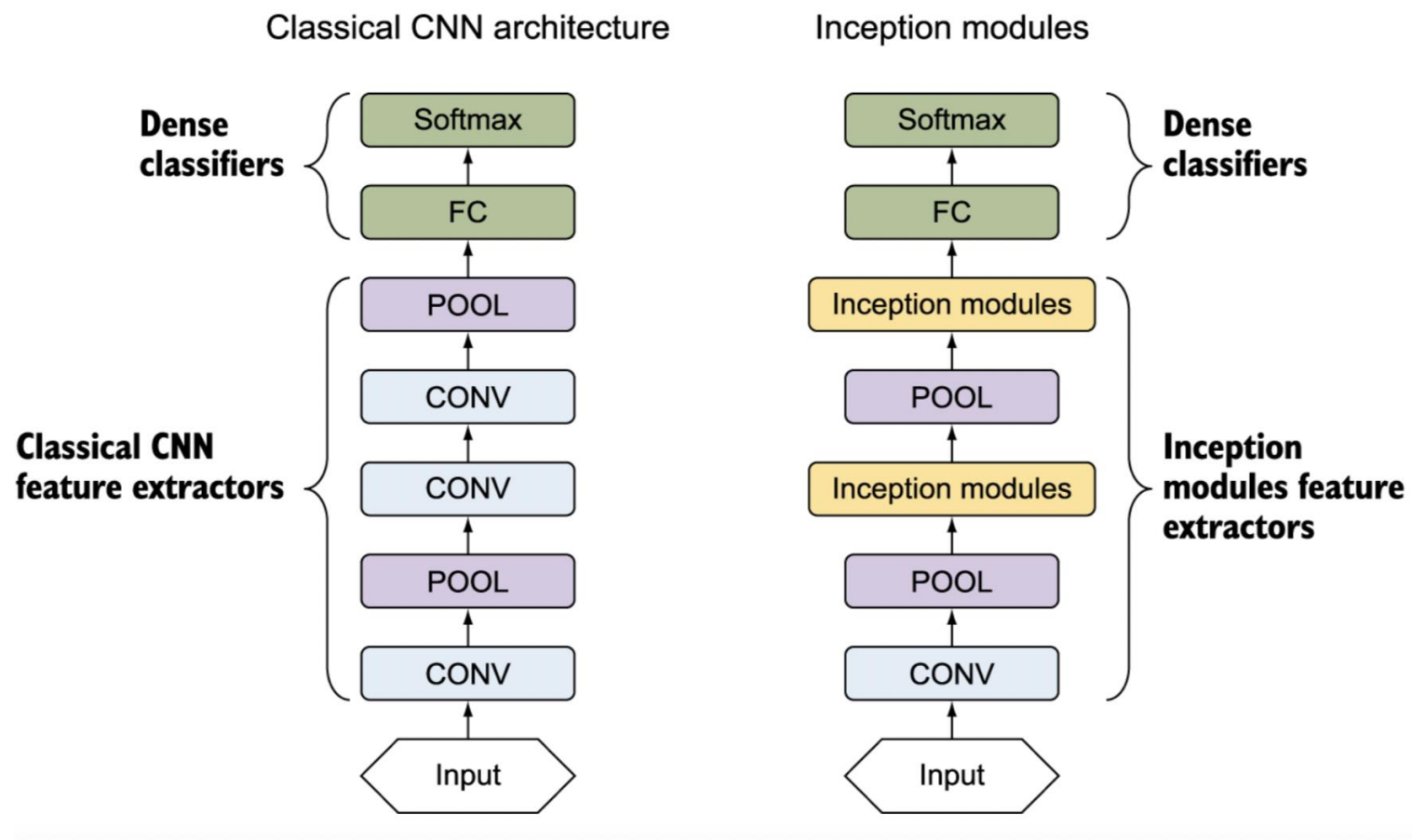
VGG Hyperparameters and performance

- Minibatch gradient descent with momentum 0.9
- Initial learning rate 0.01 with momentum 0.9
 - Reduces the learning rate as $lr/10$ when validation error stops improving
- Performance
 - Top-5 error rate : 8.1%
 - Requires fewer epochs to converge as compared to AlexNet due to smaller filter size

Architectural Decisions for Designing Networks

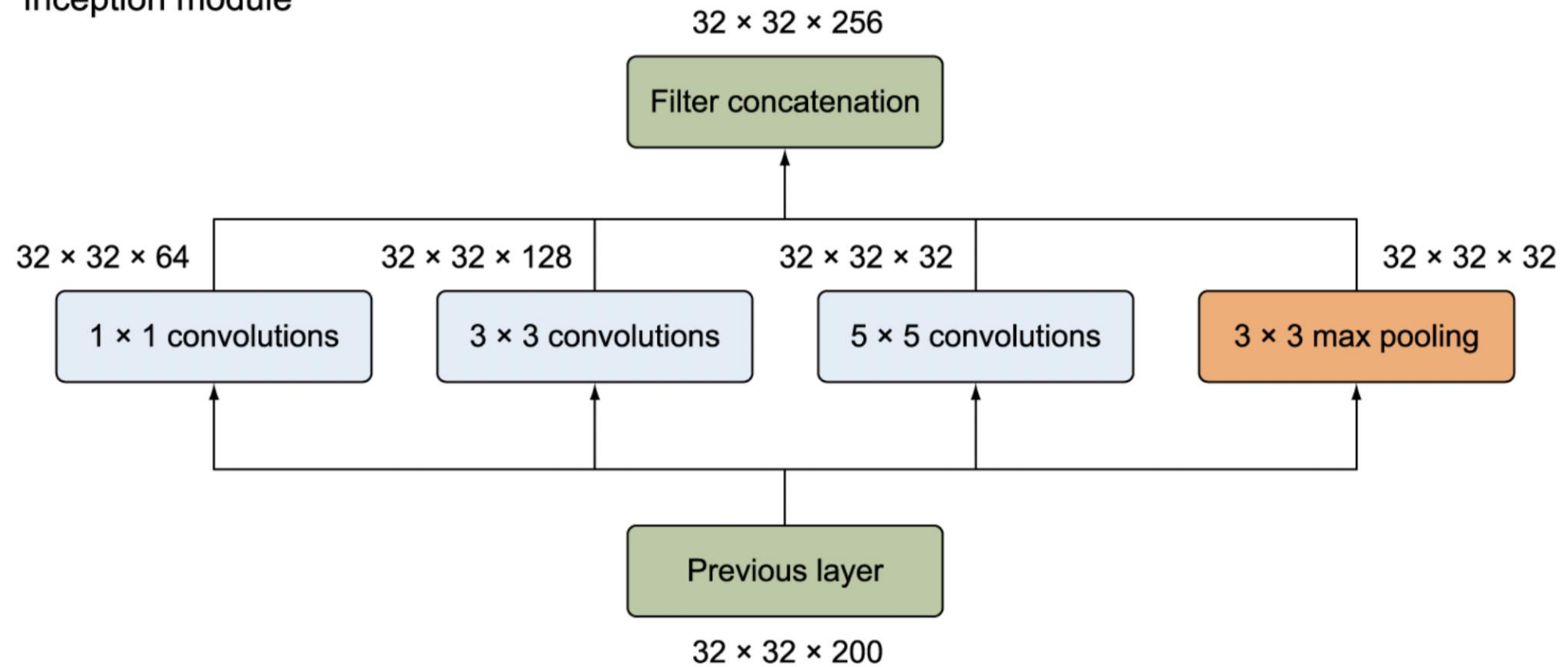
- Parameters to decide
 - Size of the kernel/filter
 - When to use pooling layer
- Approach – trial and error
- Proposed solution - Inception block
 - Includes several convolutional layers with different kernel sizes

Inception



Inception module

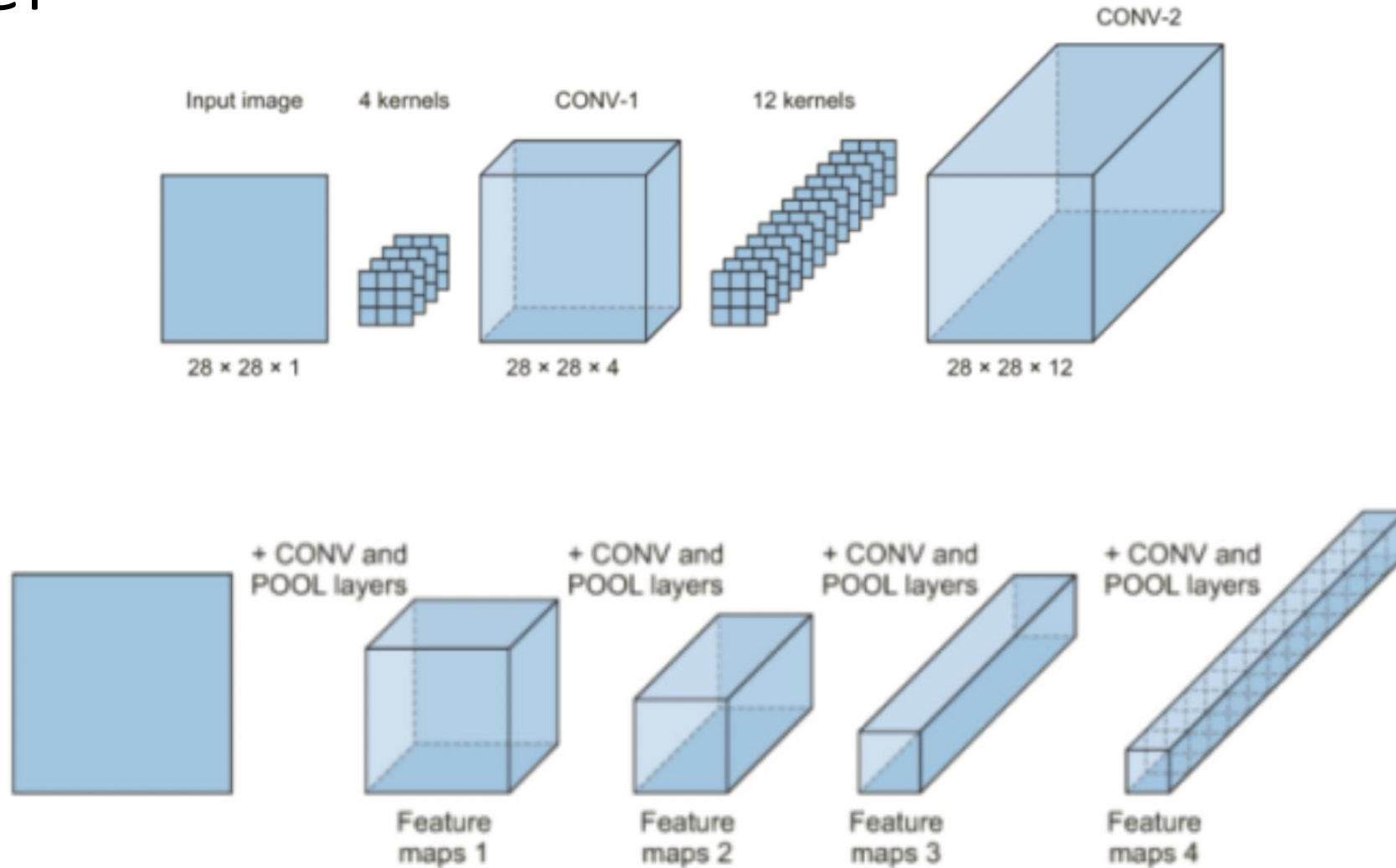
Inception module



Computation requirements with Inception block

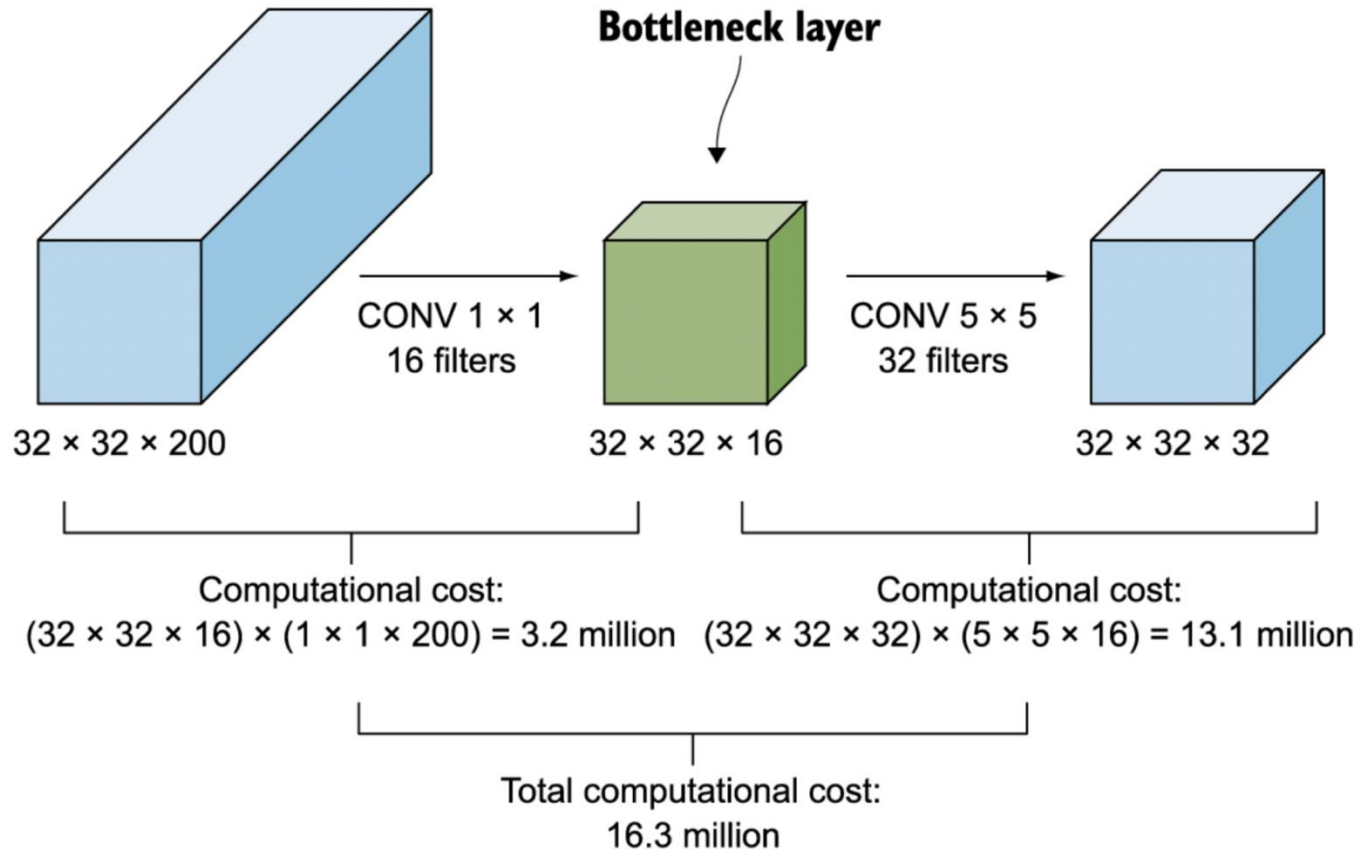
- Input = $32 \times 32 \times 200$
- Filter size = $5 \times 5 \times 32$
- Multiplications required to compute output
= $32 \times 32 \times 32 \times 5 \times 5 \times 200$
= 163 million multiplications

Visualize the network with filters and pooling layer



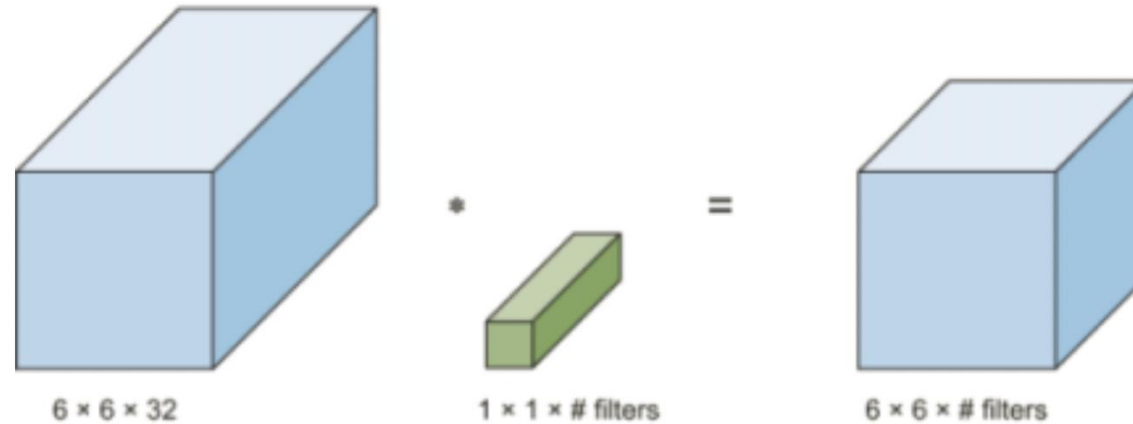
Inception with dimensionality reduction

- Reduces required computations
- When used moderately, the representation size can be reduced significantly
- Allows visual information to be processed at different scales. The next stage can aggregate the information from different scales simultaneously

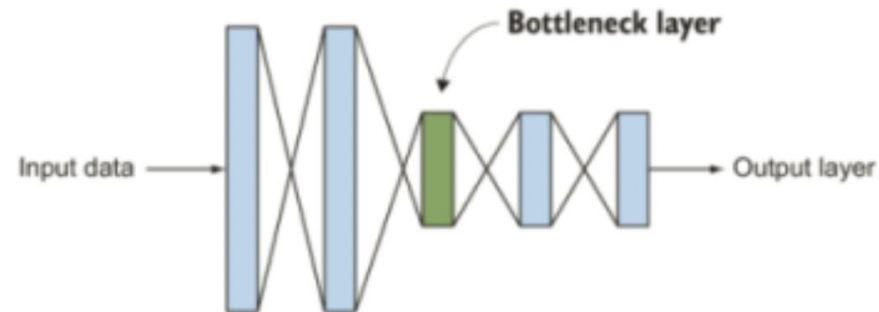


1x1 convolutional layer

1×1 conv layers preserve the spatial dimensions but change the depth.

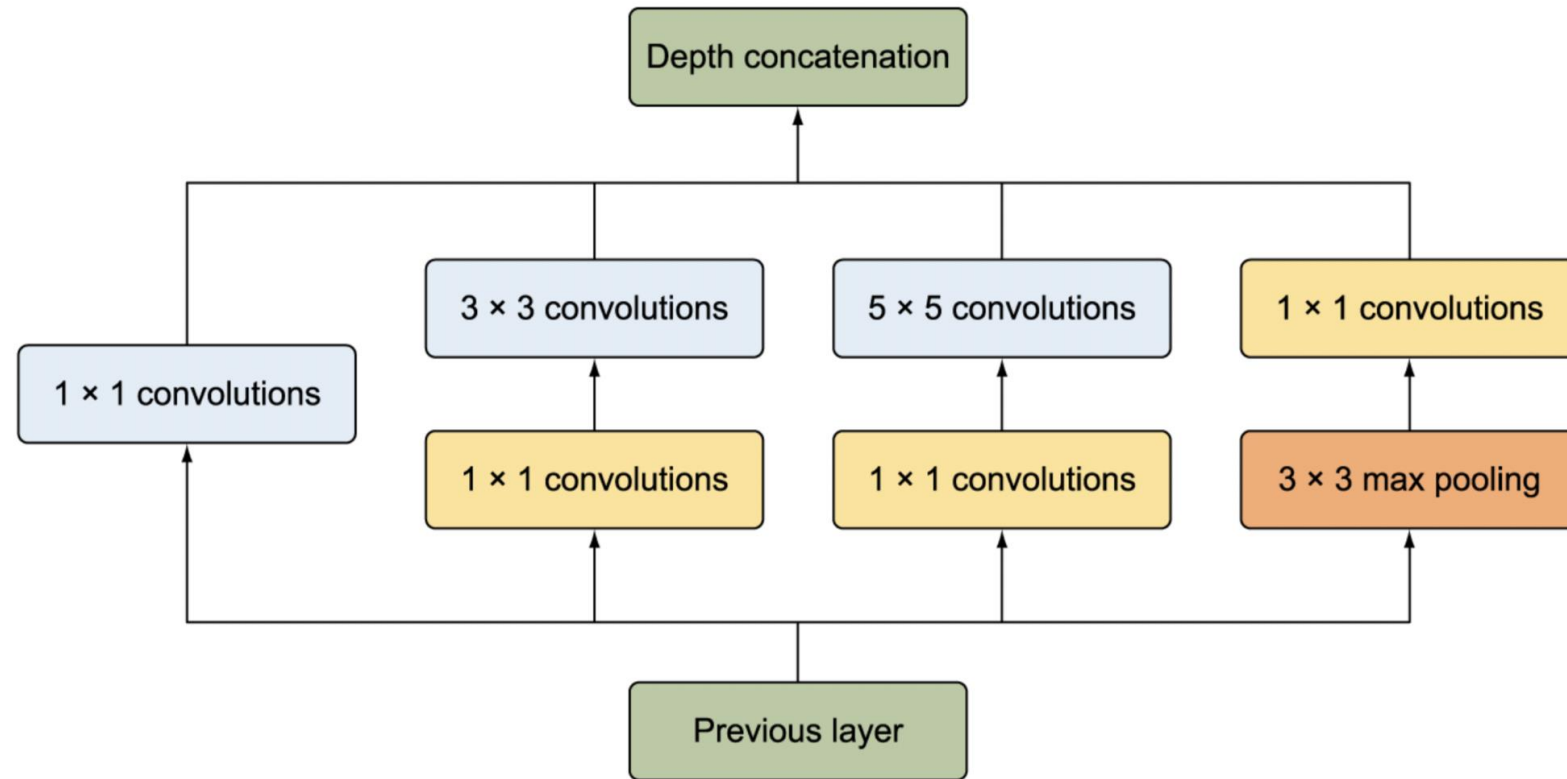


1×1 convolutional layers are called bottleneck layers.

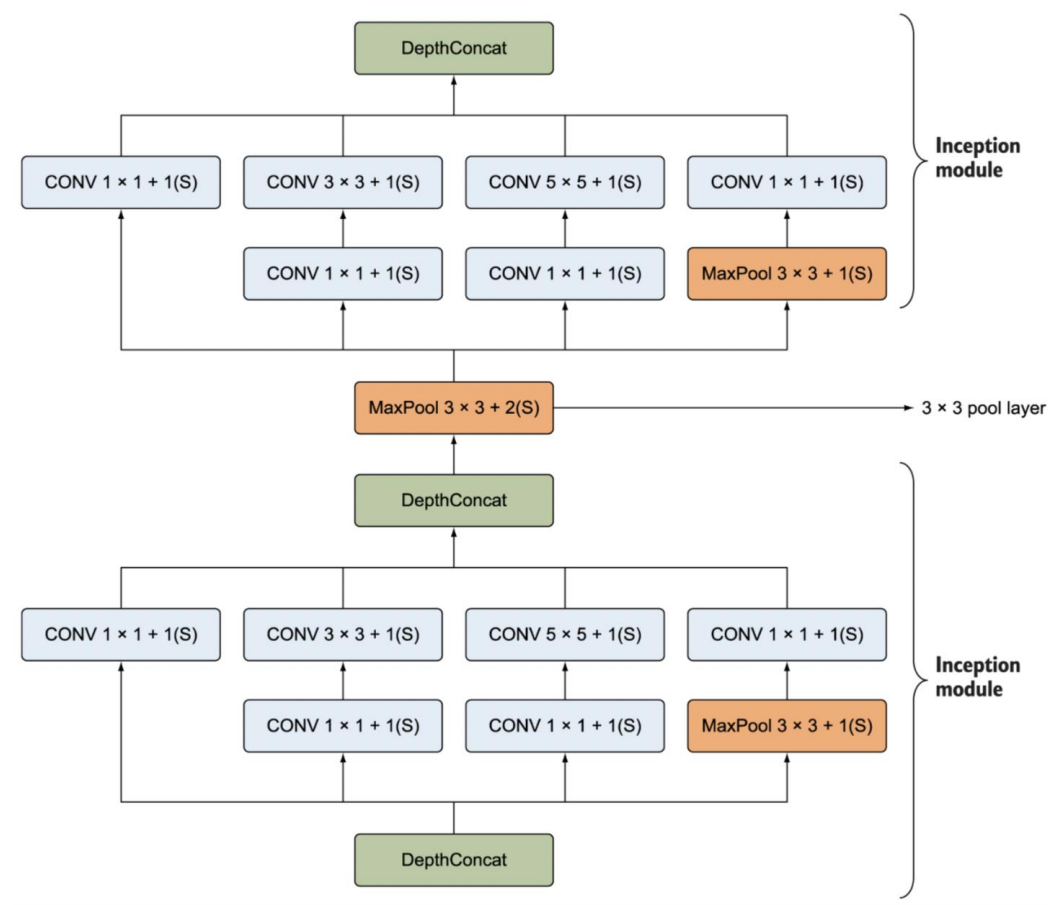


Inception Architecture

Inception module with dimensionality reduction

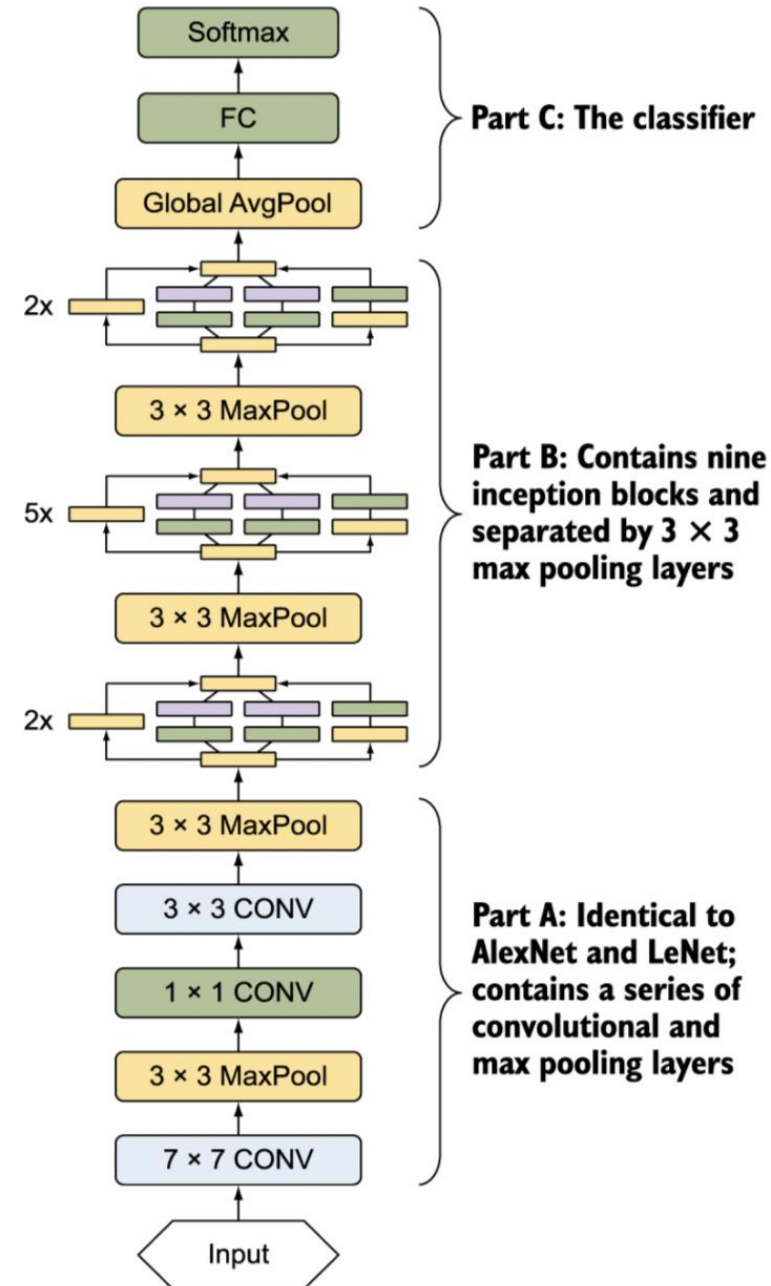


Inception Architecture



GoogleNet Architecture

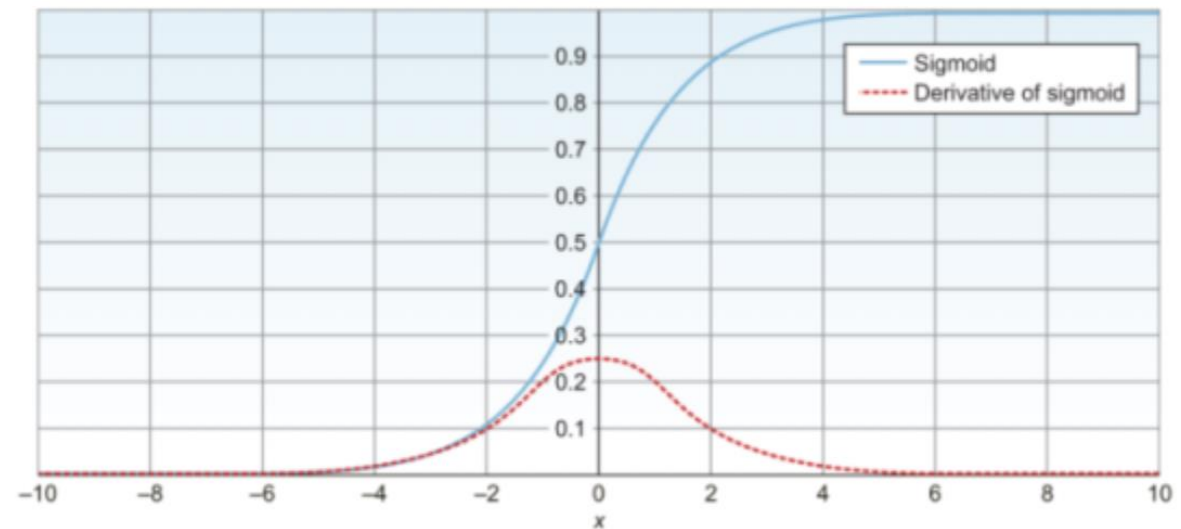
- Uses SGD with 0.9 momentum
- Uses fixed learning rate decay of 4% after every 8 epochs
- Top-5 error rate = 6.67%



Vanishing/exploding gradients

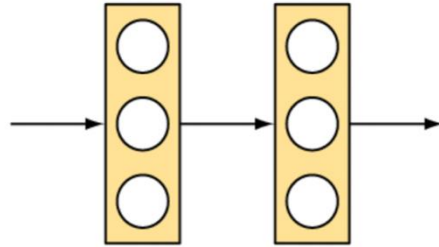
- Large change in input causes very small change in output
- During backpropagation, in order to study the effect of change in weight value on the output loss, we use the chain rule. It multiplies the derivative of loss w.r.t w from last layer moving towards the initial layers

The vanishing gradient problem: a large change in the input of the sigmoid function causes a negligible change in the output.

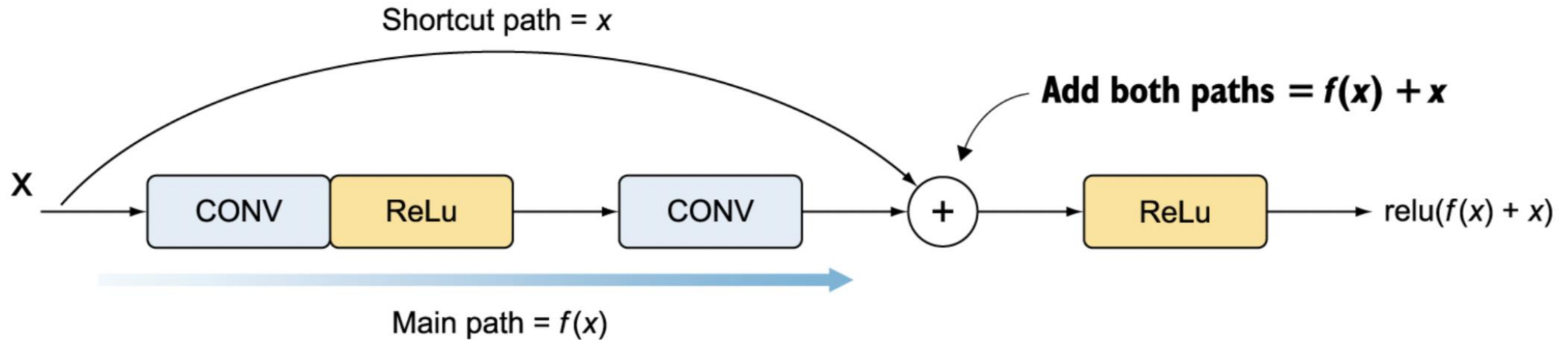
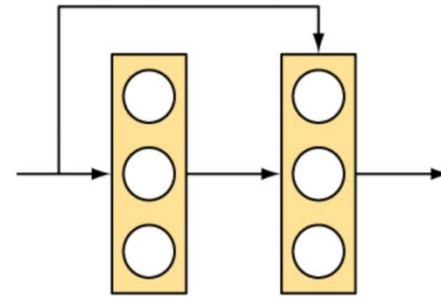


Skip connections

Without skip connection

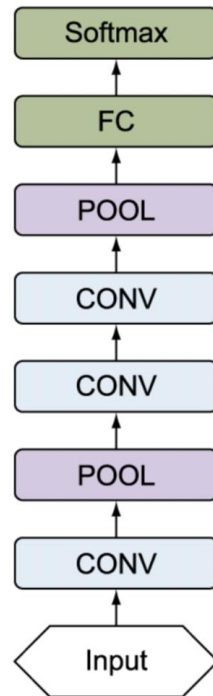


With skip connection

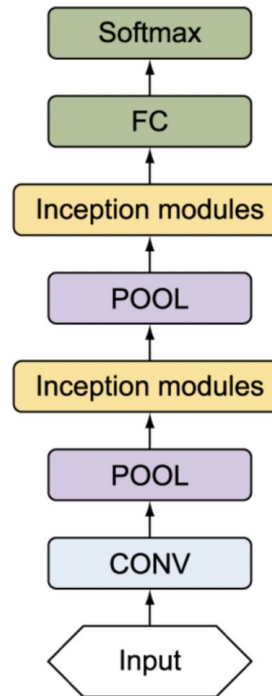


Comparison of different architectures

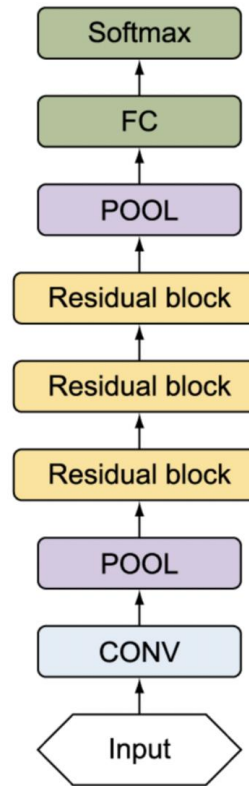
Classical CNN architecture



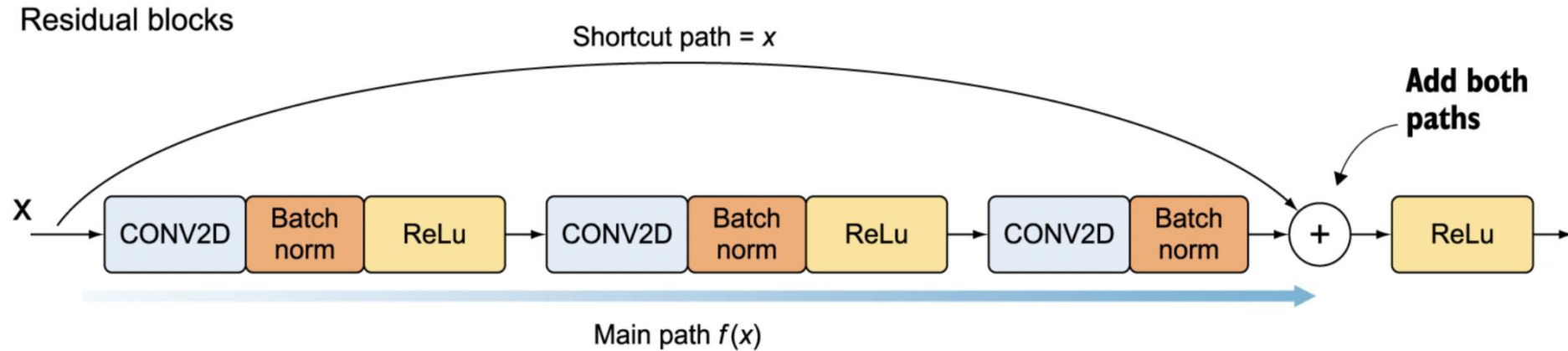
Inception modules



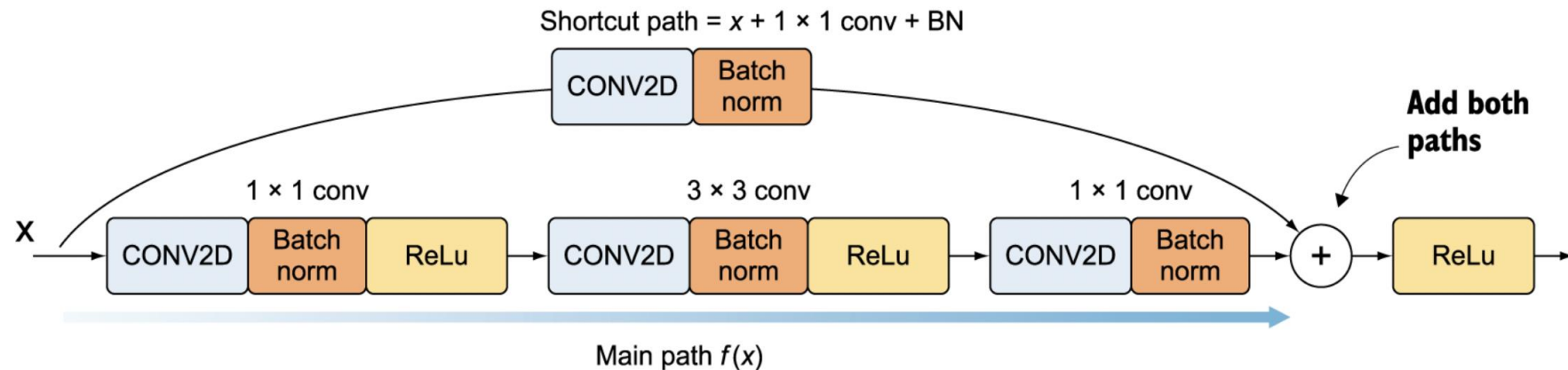
Residual blocks



Residual block



Bottleneck residual block with reduce shortcut



ResNet performance

- Uses mini-batch GD with momentum of 0.9
- Initial learning rate is 0.1 and then decreased it by a factor of 10 when the validation error stopped improving
- Top-5 error rate = 4.49%

Summary

- Classical CNN architectures have the same classical architecture of stacking convolutional and pooling layers on top of each other with different configurations for their layers.
- LeNet consists of five weight layers: three convolutional and two fully connected layers, with a pooling layer after the first and second convolutional layers.
- AlexNet is deeper than LeNet and contains eight weight layers: five convolutional and three fully connected layers.
- VGGNet solved the problem of setting up the hyperparameters of the convolutional and pooling layers by creating a uniform configuration for them to be used across the entire network.
- Inception tried to solve the same problem as VGGNet: instead of having to decide which filter size to use and where to add the pooling layer, Inception says, "Let's use them all."
- ResNet followed the same approach as Inception and created residual blocks that, when stacked on top of each other, form the network architecture. ResNet attempted to solve the vanishing gradient problem that made learning plateau or degrade when training very deep neural networks. The ResNet team introduced skip connections that allow information to flow from earlier layers in the network to later layers, creating an alternate shortcut path for the gradient to flow through. The fundamental breakthrough with ResNet was that it allowed us to train extremely deep neural networks with hundreds of layers.

Reference

- Book – Deep Learning for Vision Systems by Mohamad Elgendy, Manning Publication, November 2020